

# Voice-Based Concept Understanding Analyser Project

## Description:

The Voice-Based Concept Understanding Analyser is an AI-powered web application that evaluates a student's conceptual understanding through voice responses. The system converts spoken audio into text using OpenAI Whisper, compares the response with the expected answer using Sentence Transformers, and generates intelligent feedback using Google Gemini AI.

The application provides an interactive Streamlit interface where users upload their voice recordings. After processing, the system displays the transcript, semantic similarity score, AI-generated feedback, evaluation results, and stores the information in an SQLite database. It also generates downloadable PDF reports for future reference.

## Scenarios

### Scenario 1

A student answers a programming question by speaking into a microphone. The system converts the speech into text, compares it with the reference answer, calculates the similarity score, and provides personalized AI feedback.

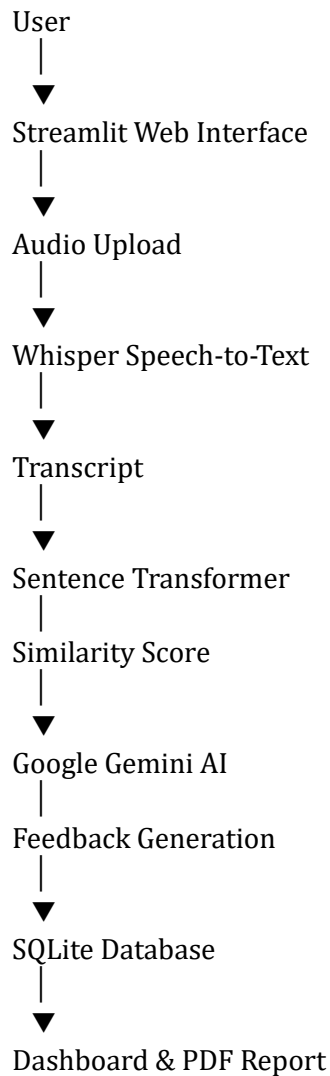
### Scenario 2

A faculty member evaluates multiple students by uploading their voice responses. The system automatically analyzes conceptual understanding and generates detailed evaluation reports.

### Scenario 3

An online learning platform integrates the application to assess learners through spoken answers instead of traditional written examinations.

## Technical Architecture:



### Description:

The application follows a modular AI-based architecture. The frontend is developed using **Streamlit**, providing an intuitive user interface for uploading voice recordings. The backend integrates **OpenAI Whisper** for speech transcription, **Sentence Transformers** for semantic similarity analysis, and **Google Gemini AI** for intelligent feedback generation. All evaluation results are stored in **SQLite**, and PDF reports are generated for documentation and future analysis.

**(Note: If you are using database in your project definitely you need to add ER Diagram along with description)**

### Pre-requisites:

- Python 3.10 or above
- Streamlit
- OpenAI Whisper
- Sentence Transformers
- Google Gemini API Key
- SQLite
- SQLAlchemy
- ReportLab
- FFmpeg
- Git
- VS Code

## **Project Workflow:**

### **Milestone 1: Environment Setup**

- Install Python and required libraries.
- Configure Google Gemini API Key.
- Install FFmpeg for Whisper.
- Set up the Streamlit project.

### **Milestone 2: Speech Recognition**

- Upload voice recordings.
- Convert speech into text using Whisper.
- Display the transcript.

### **Milestone 3: AI Evaluation**

- Compare student answers using Sentence Transformers.
- Calculate semantic similarity scores.
- Generate evaluation results.

## Milestone 4: AI Feedback

- Send transcript to Google Gemini.
- Generate personalized feedback.
- Suggest improvements

## Milestone 5: Database & Reports

- Store results in SQLite.
- Generate PDF reports.
- Display dashboard statistics.
- Maintain evaluation history.

## Milestone 6: Conclusion

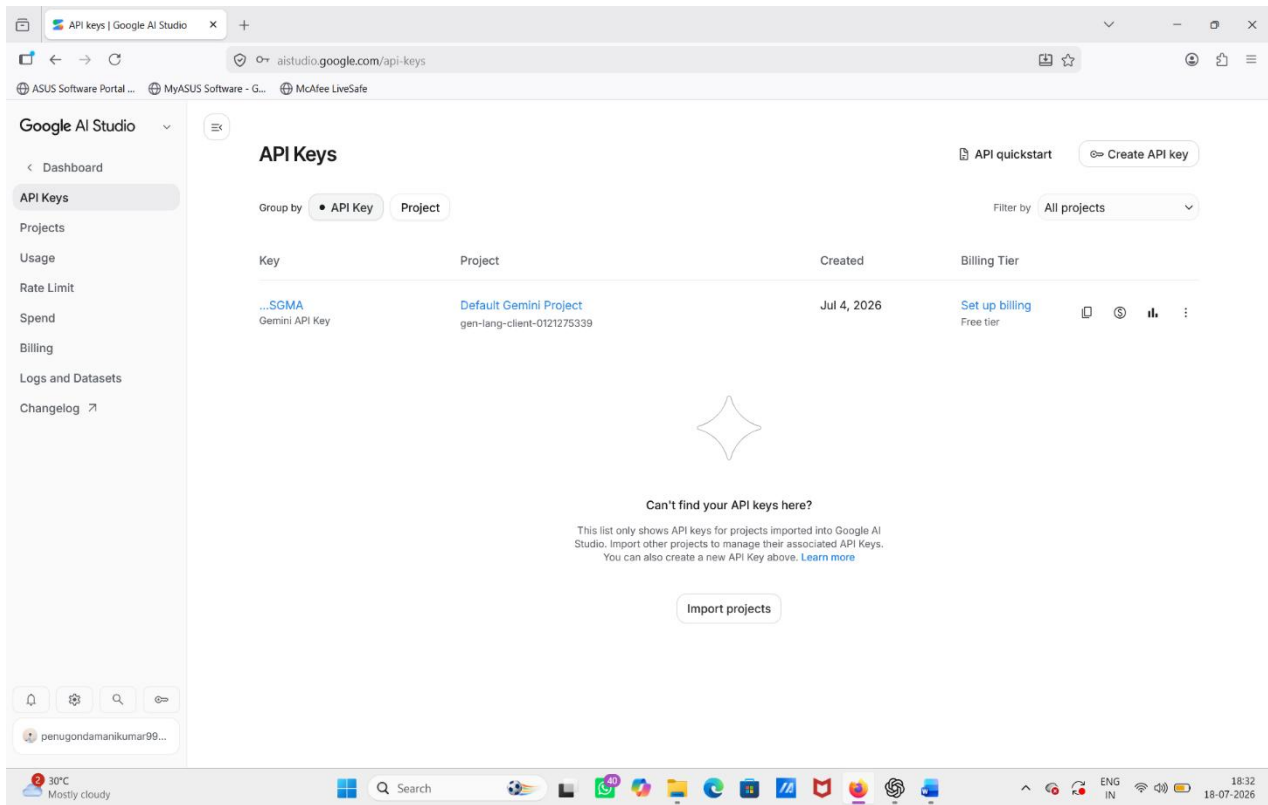
The Voice-Based Concept Understanding Analyser successfully combines speech recognition, natural language processing, semantic similarity analysis, and generative AI to automate concept evaluation. The application provides accurate assessments, intelligent feedback, detailed reports, and an intuitive dashboard, making it a valuable tool for modern educational environments.

# Milestone 1: Environment Setup and AI Model Selection

In this milestone, the development environment was configured and the AI technologies required for speech transcription, semantic similarity analysis, and intelligent feedback generation were selected. The project integrates OpenAI Whisper, Sentence Transformers, Google Gemini AI, Streamlit, SQLite, and ReportLab.

## Activity 1.1: Configure Google Gemini API Key

- Create a Google AI Studio account.
  - Generate a Gemini API Key.
  - Create a .env file in the project root.
  - Store the API key securely.
  - Load the API key using Python dotenv.
- 
- **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select “API Keys” from the dropdown menu, or go directly to the [API Keys page](#).



API Keys

Group by: **API Key** Project

Filter by: All projects

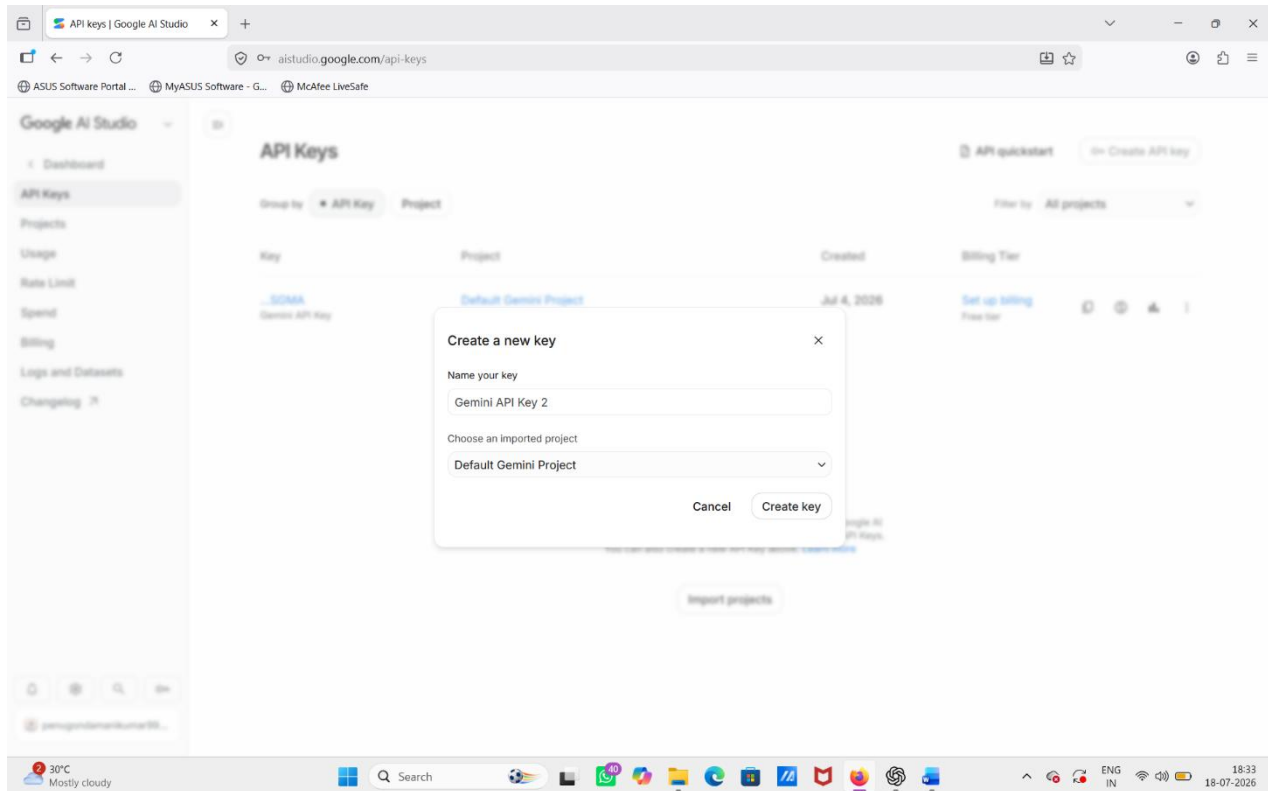
| Key                       | Project                                              | Created     | Billing Tier                |
|---------------------------|------------------------------------------------------|-------------|-----------------------------|
| ...SCMA<br>Gemini API Key | Default Gemini Project<br>gen-lang-client-0121275339 | Jul 4, 2026 | Set up billing<br>Free tier |

Can't find your API keys here?

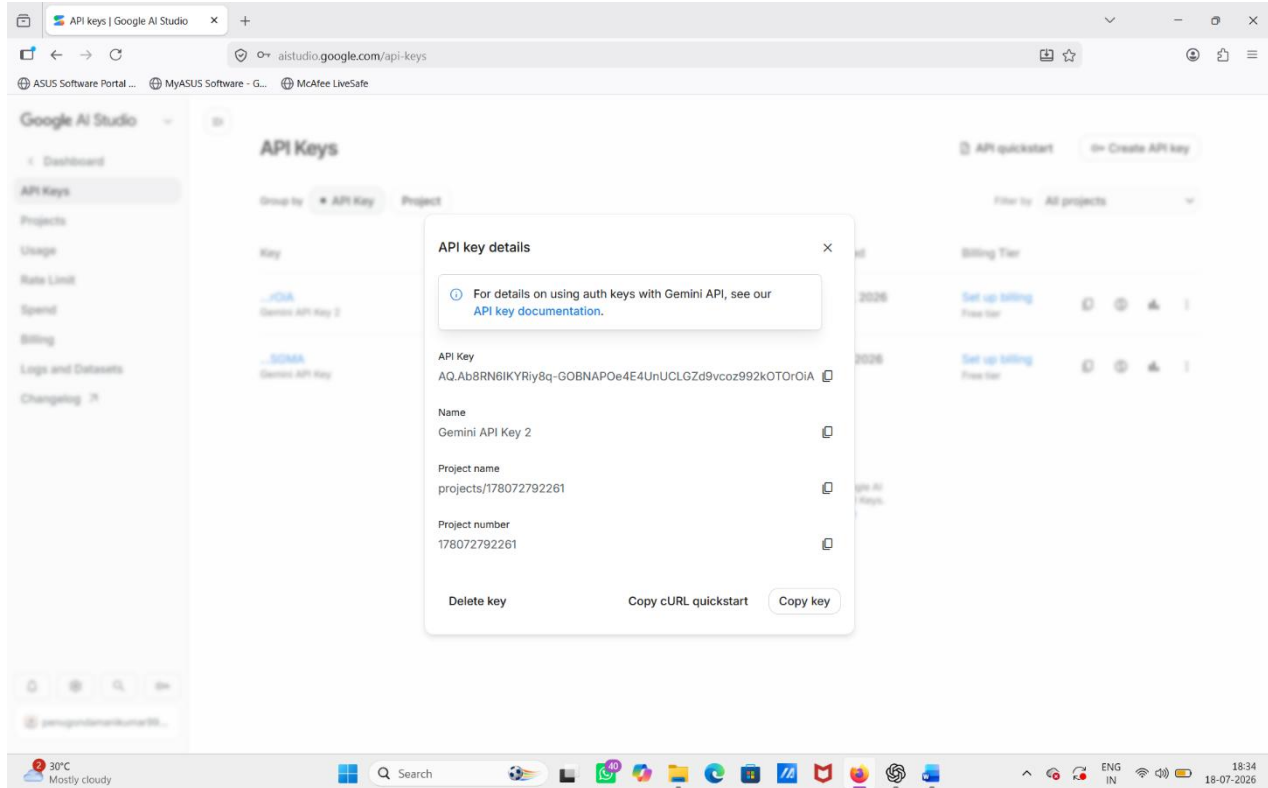
This list only shows API keys for projects imported into Google AI Studio. Import other projects to manage their associated API Keys. You can also create a new API Key above. [Learn more](#)

Import projects

- **Step 3 — Create a New API Key:** Click the “Create API Key” button. Give your key a descriptive name (e.g., captionai-key) so it is easy to identify later



- **Step 4 — API Key:** Now give a “Display Name” and “Expiration” is optional, and Click on “Submit”. The you will get your API key, Copy it immediately and store it in a safe place you will not be able to view it again after closing the dialog.



- **Step 5 — Add the Key to Your Project:** Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below:  
GROQ\_API\_KEY=your\_copied\_api\_key\_here
- **Step 6 — Verify the Key is Loaded:** The app.py file loads this key automatically at startup using python-dotenv: from dotenv import load\_dotenv; load\_dotenv()  
client = Groq(api\_key=os.environ.get("GEMINI\_API\_KEY"))
- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:  
.env

## Activity 1.2: Research and Select the Appropriate Generative AI Model

Install the required Python libraries:

```
pip install -r requirements.txt
```

Main libraries used:

- Streamlit
- OpenAI Whisper
- Sentence Transformers
- Google Generative AI

- SQLAlchemy
- ReportLab
- python-dotenv

### Activity 1.3: Define the Architecture of the Application

The project follows a modular architecture consisting of:

- Streamlit Frontend
- Whisper Speech-to-Text Module
- Sentence Transformer Similarity Engine
- Google Gemini Feedback Generator
- SQLite Database
- PDF Report Generator

### Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv  
D:\projects>"d:\projects\venv\Scripts\activate"  
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install python-dotenv:** Install dotenv support for secure API key management:
- streamlit run frontend/streamlit\_app.py

## Milestone 2: Core Functionalities Development

### Activity 2.1: Develop Core Content Generation Features

Implement the three core generation features Captions, Hashtags, and CTAs as a unified AI response driven by a single structured prompt.

- **Caption Generator:** Generates three unique, platform-appropriate captions. For Instagram/Casual tone, includes relevant emojis and conversational language. For LinkedIn/Professional tone, produces insight-led, value-driven copy.
- **Hashtag Suggester:** Produces ten hashtags per request a curated mix of high-reach popular tags and targeted niche tags relevant to the product or campaign.
- **CTA Generator:** Returns three short (under 10 words), punchy, action-oriented call-to-action phrases calibrated to the selected platform and tone.

## Activity 2.2: Implement the Flask Backend

### Define Routes in Flask:

- Set up routes in app.py for the home page and content generation endpoint.

### Process User Input:

- Create a form in index.html for users to submit their product/campaign information, platform, and tone.
- Use Flask's `request.get_json()` to retrieve JSON data submitted from the frontend via AJAX.
- Validate that `product_info` is non-empty and within the 2000-character limit before processing.

### Integrate API Calls:

- Within the `/generate` route handler, call the `build_prompt()` function to construct a structured prompt combining tone descriptions and platform-specific tips.
- Submit the prompt to the Groq API using `client.chat.completions.create()` with the `llama-3.3-70b-versatile` model.
- Parse and validate the JSON response using `extract_json()`, which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.

## Milestone 3: Writing the Main Application Logic

- Create the main Streamlit application (streamlit\_app.py) to provide the user interface.
- Allow users to upload or record an audio file.
- Validate the uploaded audio format before processing.
- Call the Whisper module to convert speech into text.
- Pass the transcript to the Semantic Similarity module.
- Send the transcript and similarity score to the Gemini AI module for evaluation.
- Store the evaluation results in the SQLite database.
- Display the transcript, similarity score, AI feedback, and final evaluation on the dashboard.

### Define the Core Routes in app.py:

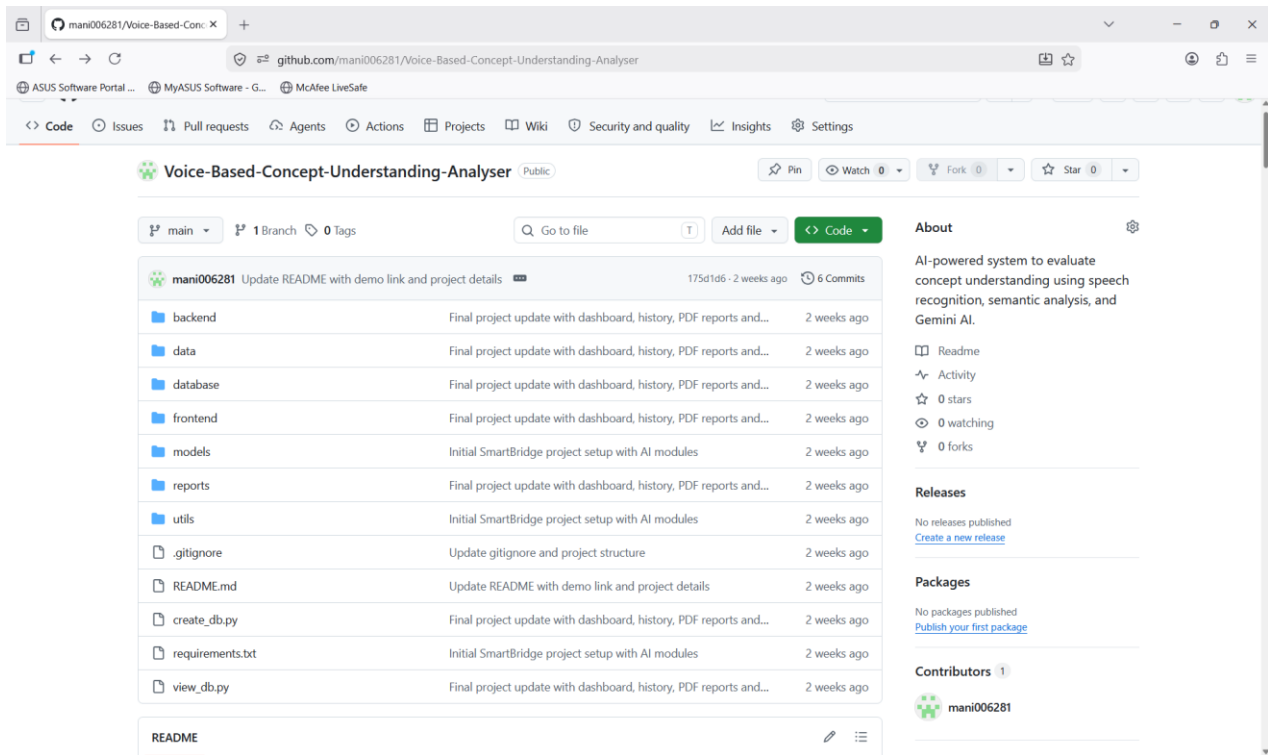
- Establish routes for CaptionAI's two endpoints, each serving a distinct purpose:
  - **/** — Home page route that renders the main index.html interface

```
@app.route("/")
def index():
    return render_template("index.html")
```

- **/generate** — POST endpoint that accepts JSON input, invokes the AI, and returns structured content

### Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads product\_info, platform, and tone from the incoming JSON body using request.get\_json().
- Apply input validation: return a 400 error if product\_info is empty or exceeds 2000 characters.
- Route AJAX requests from the frontend to the /generate endpoint, which processes data and returns a JSON response.



The screenshot shows a GitHub repository page for "Voice-Based-Concept-Understanding-Analyser" by user mani006281. The repository is public and has 6 commits. The file list includes folders for backend, data, database, frontend, models, reports, and utils, as well as files like .gitignore, README.md, create\_db.py, requirements.txt, and view\_db.py. The README file is selected, showing its content. The right sidebar contains sections for About, Releases, Packages, and Contributors.

**Voice-Based-Concept-Understanding-Analyser** Public

175d1d6 - 2 weeks ago 6 Commits

| File/Folder      | Description                                                      | Time        |
|------------------|------------------------------------------------------------------|-------------|
| backend          | Final project update with dashboard, history, PDF reports and... | 2 weeks ago |
| data             | Final project update with dashboard, history, PDF reports and... | 2 weeks ago |
| database         | Final project update with dashboard, history, PDF reports and... | 2 weeks ago |
| frontend         | Final project update with dashboard, history, PDF reports and... | 2 weeks ago |
| models           | Initial SmartBridge project setup with AI modules                | 2 weeks ago |
| reports          | Final project update with dashboard, history, PDF reports and... | 2 weeks ago |
| utils            | Initial SmartBridge project setup with AI modules                | 2 weeks ago |
| .gitignore       | Update gitignore and project structure                           | 2 weeks ago |
| README.md        | Update README with demo link and project details                 | 2 weeks ago |
| create_db.py     | Final project update with dashboard, history, PDF reports and... | 2 weeks ago |
| requirements.txt | Initial SmartBridge project setup with AI modules                | 2 weeks ago |
| view_db.py       | Final project update with dashboard, history, PDF reports and... | 2 weeks ago |

**About**

AI-powered system to evaluate concept understanding using speech recognition, semantic analysis, and Gemini AI.

**Releases**

No releases published  
[Create a new release](#)

**Packages**

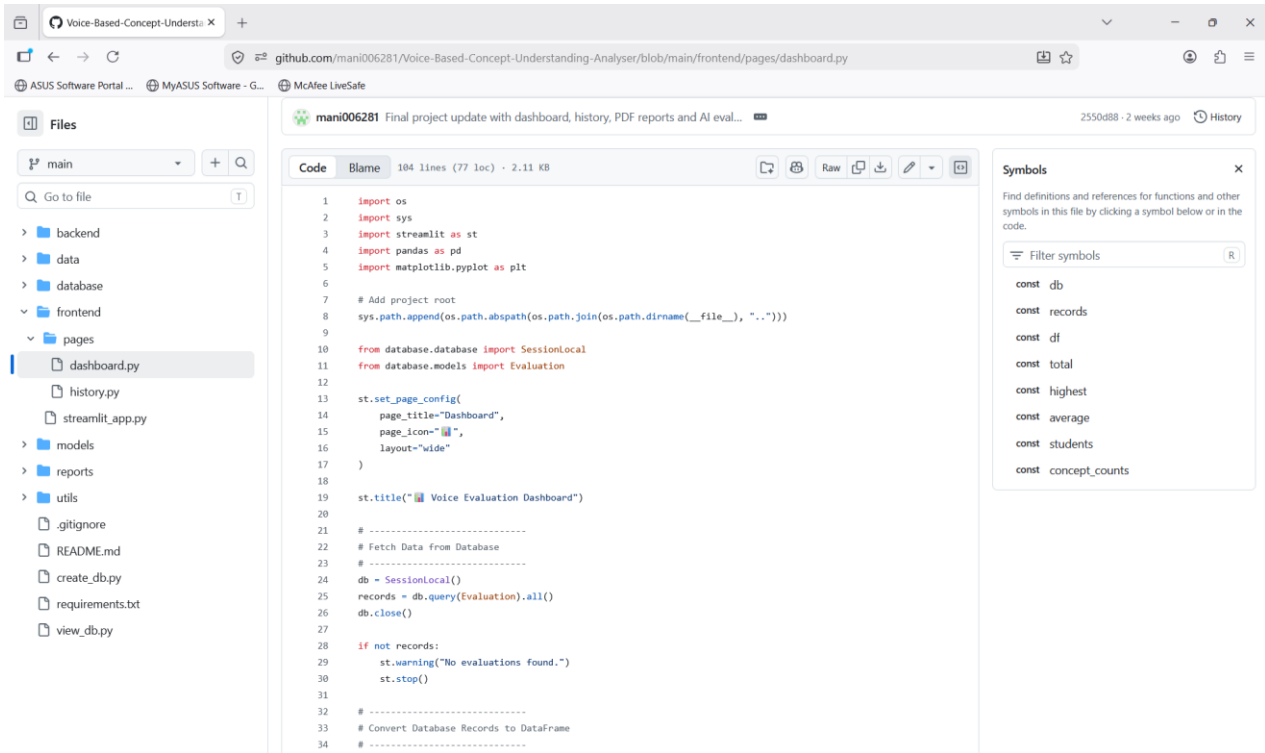
No packages published  
[Publish your first package](#)

**Contributors** 1

mani006281

## Define Prompt Engineering Helpers:

- **TONE\_DESCRIPTIONS** dictionary: maps professional, casual, and promotional tones to detailed copywriting instructions embedded in the prompt.



The screenshot shows a GitHub repository for 'Voice-Based-Concept-Understanding-Analyser'. The file 'dashboard.py' is selected, showing 104 lines of code. The code imports libraries like os, sys, streamlit, pandas, and matplotlib. It sets up a session local database, fetches data, and displays it in a Streamlit dashboard titled 'Voice Evaluation Dashboard'.

```

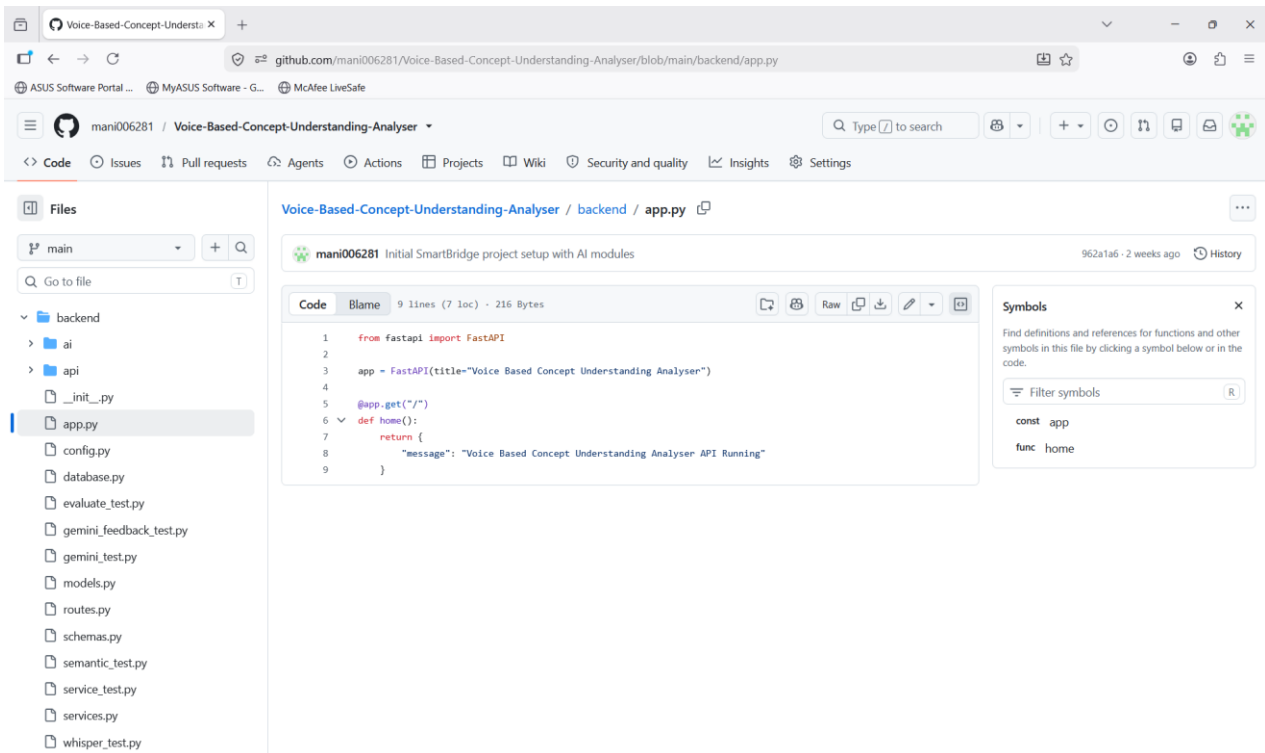
1 import os
2 import sys
3 import streamlit as st
4 import pandas as pd
5 import matplotlib.pyplot as plt
6
7 # Add project root
8 sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
9
10 from database.database import SessionLocal
11 from database.models import Evaluation
12
13 st.set_page_config(
14     page_title="Dashboard",
15     page_icon="📊",
16     layout="wide"
17 )
18
19 st.title("📊 Voice Evaluation Dashboard")
20
21 # -----
22 # Fetch Data from Database
23 # -----
24 db = SessionLocal()
25 records = db.query(Evaluation).all()
26 db.close()
27
28 if not records:
29     st.warning("No evaluations found.")
30     st.stop()
31
32 # -----
33 # Convert Database Records to DataFrame
34 # -----

```

The Symbols panel on the right lists the following symbols:

- const db
- const records
- const df
- const total
- const highest
- const average
- const students
- const concept\_counts

- **PLATFORM\_TIPS** dictionary: maps instagram and linkedin to platform-specific content guidelines (character limits, style expectations).



The screenshot shows the same GitHub repository, but the file 'app.py' is selected, showing 9 lines of code. The code imports FastAPI and sets up a FastAPI application with a single endpoint 'home' that returns a message.

```

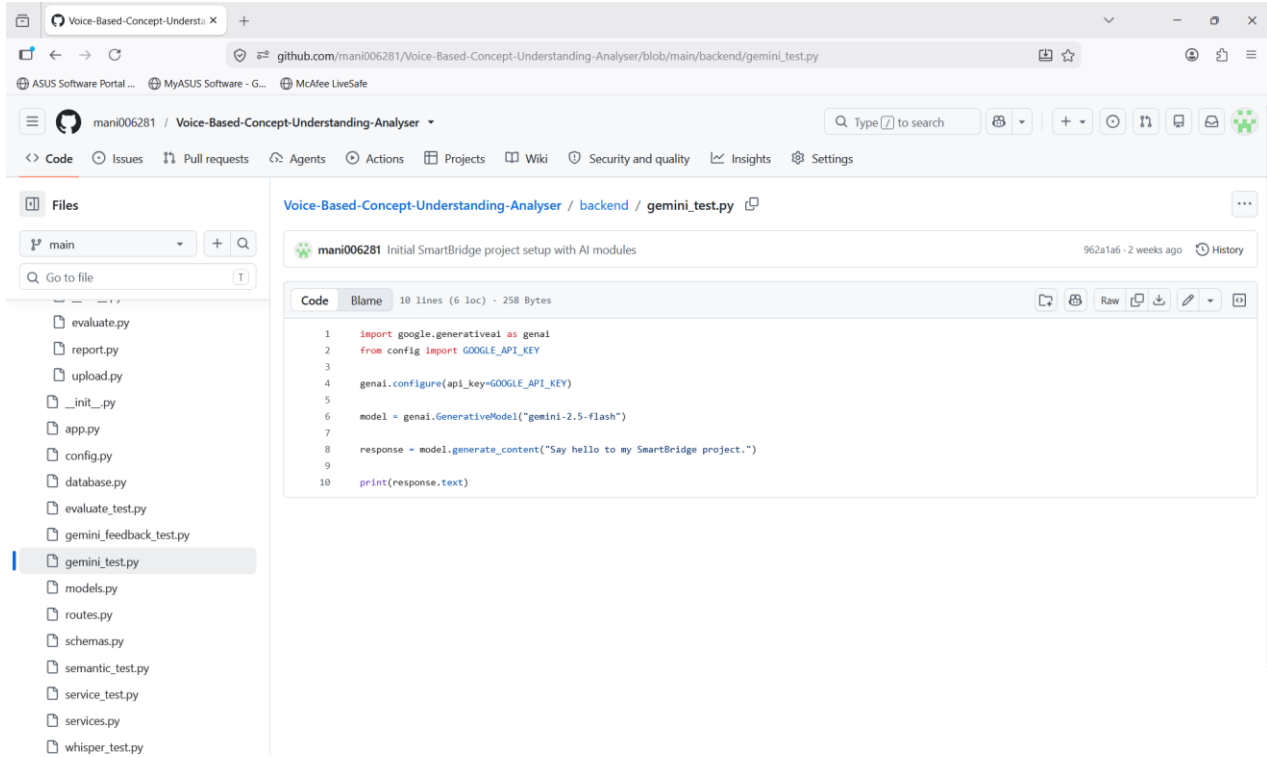
1 from fastapi import FastAPI
2
3 app = FastAPI(title="Voice Based Concept Understanding Analyser")
4
5 @app.get("/")
6 def home():
7     return {
8         "message": "Voice Based Concept Understanding Analyser API Running"
9     }

```

The Symbols panel on the right lists the following symbols:

- const app
- func home

- **build\_prompt()**: assembles a structured prompt string from the product info, selected platform tip, and tone description, instructing the model to return strictly valid JSON.



The screenshot shows a GitHub repository named 'Voice-Based-Concept-Understanding-Analyser' by user 'mani006281'. The file 'gemini\_test.py' is selected in the file explorer on the left. The code in the main pane is as follows:

```

1  import google.generativeai as genai
2  from config import GOOGLE_API_KEY
3
4  genai.configure(api_key=GOOGLE_API_KEY)
5
6  model = genai.GenerativeModel("gemini-2.5-flash")
7
8  response = model.generate_content("Say hello to my SmartBridge project.")
9
10 print(response.text)
  
```

## Milestone 4: Frontend Development

Milestone 4 focuses on developing an intuitive and user-friendly interface for the Voice-Based Concept Understanding Analyser using Streamlit. The interface enables users to upload or record voice responses, evaluate concept understanding, view AI-generated feedback, and access previous evaluation history through an interactive

### 4.1: Designing and Developing the User Interface

#### Set Up the Base HTML Structure:

- Design the Streamlit home page.
- Provide an audio upload/record option.
- Add a "Start Evaluation" button.
- Display the transcript after speech-to-text conversion.
- Display semantic similarity score.
- Display Gemini AI feedback.
- Show evaluation history.

- Keep the interface simple and responsive.

### Design a Responsive Layout Using CSS:

- Receive uploaded audio.
- Process audio using Whisper.
- Compute similarity score.
- Generate AI feedback using Gemini.
- Display results instantly.
- Save reports into SQLite.
- Update dashboard automatically.

### Create Separate UI Components for Each Output Type:

- **Caption Cards:** individual cards per caption, each with a one-click copy-to-clipboard button that provides visual feedback (checkmark + green highlight for 2 seconds).
- **Hashtag Chips:** rendered as pill-shaped chip elements inside a glass card, with automatic # prefix validation.
- **CTA List:** displayed as a clean unordered list inside a separate glass card with a bullhorn icon header.

## Activity 4.2: Creating Dynamic Content Rendering with JavaScript

### Handle Form Submission Asynchronously:

- Attach a submit event listener to the generator form that prevents default form submission and sends the data as a JSON POST request to /generate via the Fetch API.
- During loading, disable the submit button and show an animated CSS loader in place of the button text and arrow icon.

### Render Results Dynamically:

- Implement the renderResults(data) function in main.js, which dynamically creates and injects DOM elements for captions, hashtags, and CTAs into their respective containers.
- After rendering, automatically smooth-scroll the viewport to the results section using scrollToView().

### Restore UI State After Generation:

- In the finally block of the async handler, re-enable the submit button and restore the original button text and icon regardless of success or failure.

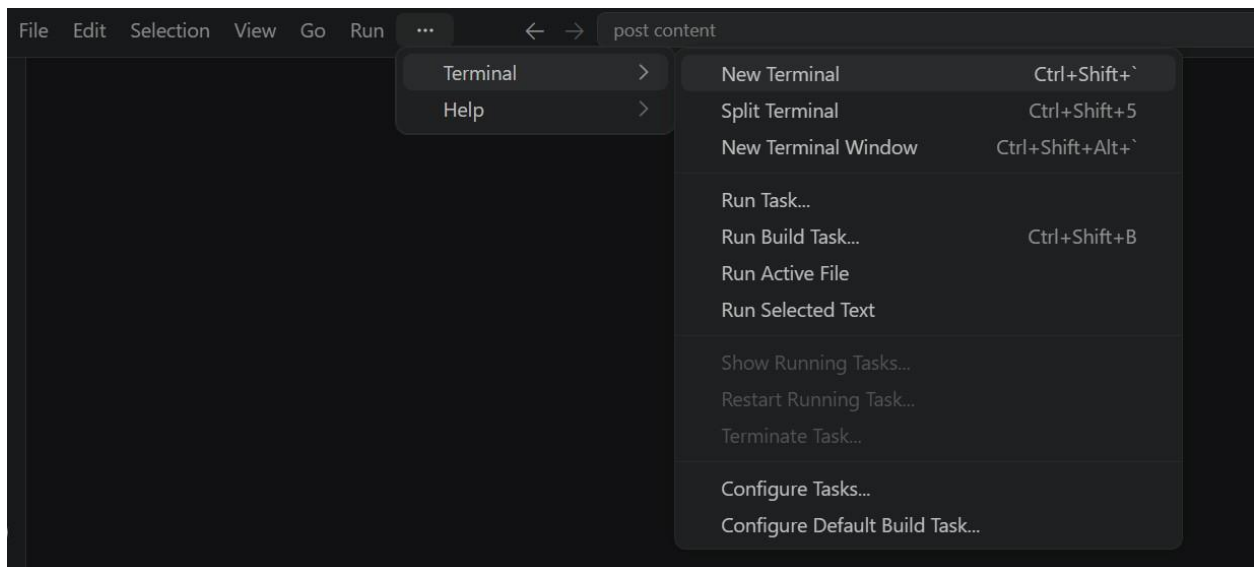
## Milestone 5: Deployment

In Milestone 5, the focus is on deploying the CaptionAI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

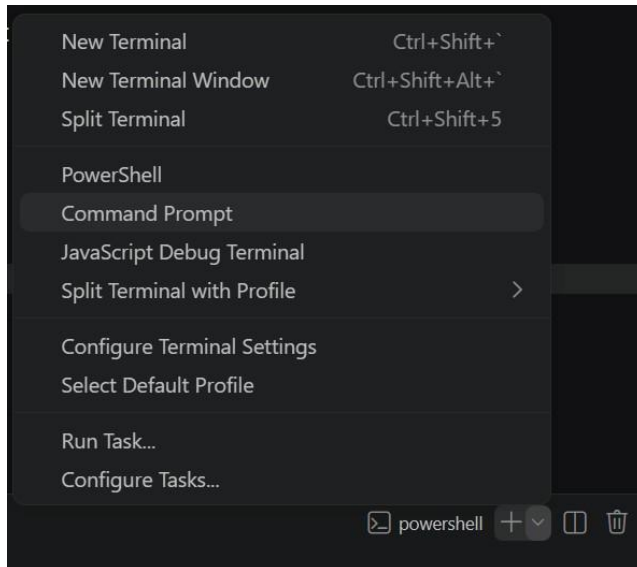
### Activity 5.1: Preparing the Application for Local Deployment

#### Set Up a Virtual Environment:

- Open a New Terminal



- Then open "Command Prompt"



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

### Configure Environment Variables:

Create a **.env** file in the project root.

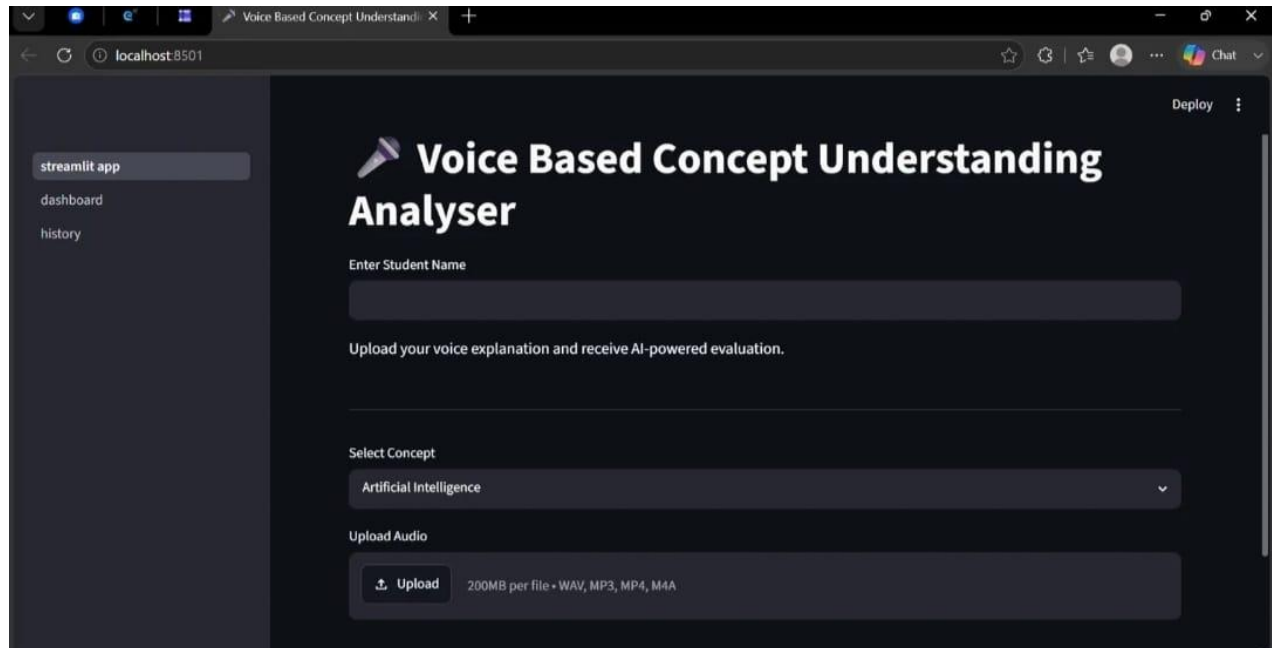
GOOGLE\_API\_KEY=AIzaSy\*\*\*\*\*

### Important Notes:

- **debug=False** is required in run\_public.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart run\_public.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart run\_public.py to get a new URL when this happens.

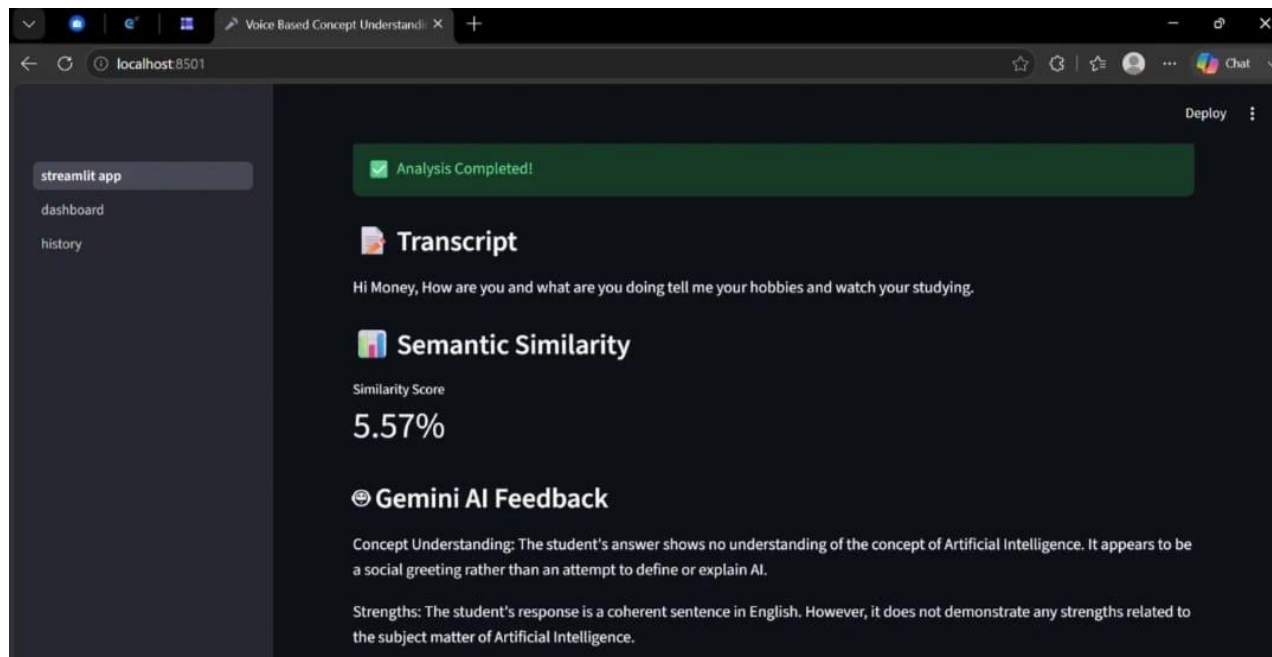
## Exploring the Website's Web Pages:

### Home / Generator Page:



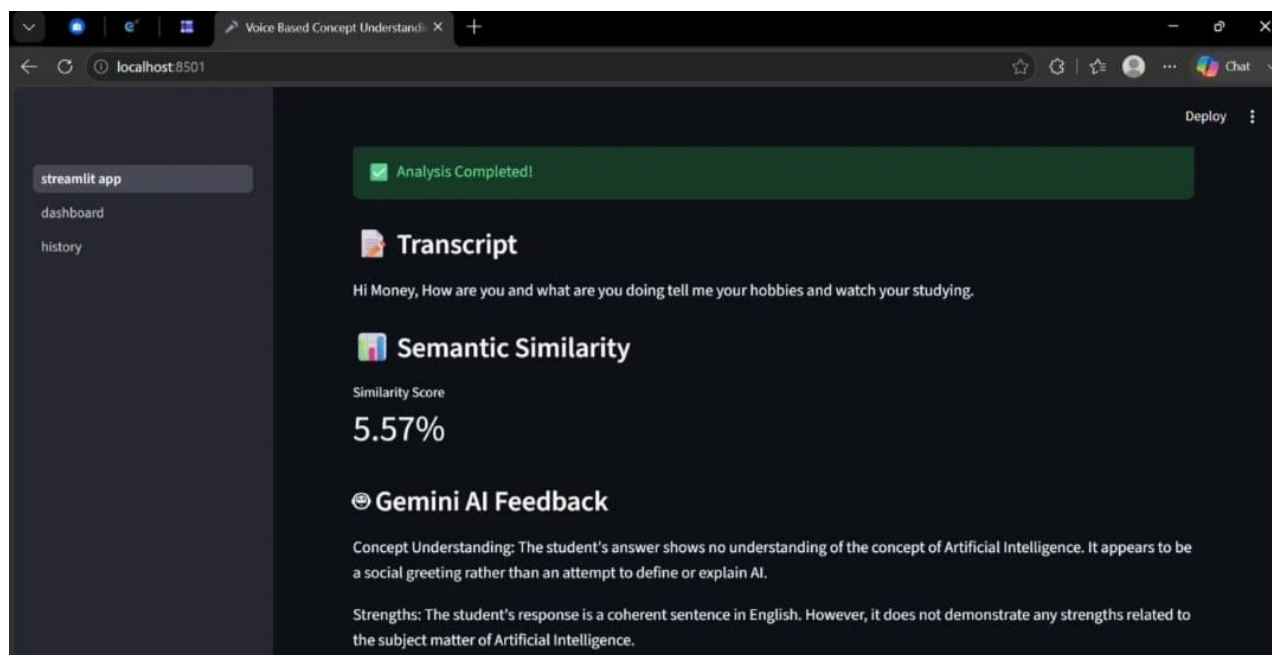
**Description:** The Home page of the **Voice Based Concept Understanding Analyser** is developed using **Streamlit** and serves as the primary interface for student interaction. Users enter their name, select a concept, and upload a voice recording. After clicking **Evaluate**, the system processes the audio using Whisper, evaluates conceptual understanding through semantic similarity, generates AI-based feedback using Gemini, and displays the results in an organized manner.

### Input Section:



**Description:** The input section allows users to enter their details, select the desired concept, and upload an audio response. Supported audio formats include WAV, MP3, and M4A. Once the audio is uploaded, the evaluation process begins automatically after the user clicks the Evaluate button.

## Results Section:



**Description:** After processing, the application displays the speech transcript, semantic similarity score, AI-generated feedback, strengths, improvement suggestions, and the overall evaluation. The results are also stored in the database for future reference and dashboard visualization.

**Conclusion:** The **Voice Based Concept Understanding Analyser** provides an intelligent solution for evaluating students' conceptual understanding through voice responses. By integrating **Whisper Speech Recognition, Sentence Transformers, Google Gemini AI, SQLite**, and **Streamlit**, the system automatically transcribes speech, measures semantic similarity with the expected answer, and generates personalized AI feedback. This project improves the learning experience by providing instant evaluation and constructive suggestions while maintaining a user-friendly interface. The modular architecture also allows future enhancements such as multilingual support, additional subject domains, and cloud deployment.